

ADP470S

Applications Development Practice 4

COMPLETE ANSWERS & CODE EXAMPLES

All Test Papers: 2021 · 2024 · 2025 + Practice Questions

Paper	Questions Covered
Practice Questions	Factorial (rec/iter), Big O, Bubble, Insertion, QuickSort
Test 1 — 2021	BST, Circular Queue, Linked Stack, Factorial
Test 1 — 2024	Factorial traces, QuickSort, HeapSort, Big O
Test 1 — 2025	Big O code excerpts, Factorial, Bubble/Quick/Heap Sort

PRACTICE QUESTIONS — Full Answers & Code

PQ1 Implement a recursive method for factorial.

[5]

Answer + Java Code

The recursive approach mirrors the mathematical definition directly. It works by having the method call itself with a smaller value each time, until it reaches the base case ($n=0$ or $n=1$).

```
public class Factorial {  
  
    /**  
     * Recursive factorial method.  
     * Base case:      n == 0 or n == 1 → return 1  
     * Recursive case: n >= 2          → return n * factorial(n-1)  
     */  
    public static int factorialRecursive(int n) {  
        if (n == 0 || n == 1) {           // BASE CASE - stop recursing  
            return 1;  
        }  
        return n * factorialRecursive(n - 1); // RECURSIVE CASE  
    }  
  
    public static void main(String[] args) {  
        System.out.println(factorialRecursive(5)); // Output: 120  
        System.out.println(factorialRecursive(0)); // Output: 1  
    }  
}
```

Execution Trace for factorial(5)

factorial(5) → calls factorial(4)

factorial(4) → calls factorial(3)

factorial(3) → calls factorial(2)

factorial(2) → calls factorial(1)

factorial(1) = 1 ← BASE CASE (returns 1)
factorial(2) = 2 × 1 = 2 (returns 2)
factorial(3) = 3 × 2 = 6 (returns 6)
factorial(4) = 4 × 6 = 24 (returns 24)
factorial(5) = 5 × 24 = 120 ← FINAL ANSWER

PQ2	Implement an iterative method for factorial.	[5]
------------	---	------------

Answer + Java Code

The iterative approach uses a simple for-loop. No method calls are made — just a loop that multiplies a running total.

public class Factorial {
/**
* Iterative factorial method.
* Uses a loop from 2 to n, multiplying a result variable each time.
*/
public static long factorialIterative(int n) {
long result = 1; // Start with 1 (0! = 1! = 1)
for (int i = 2; i <= n; i++) {
result = result * i; // Multiply running total by i
}
return result;
}
public static void main(String[] args) {
System.out.println(factorialIterative(5)); // Output: 120
System.out.println(factorialIterative(0)); // Output: 1
}
}

Step	Action / Explanation	Array / State
------	----------------------	---------------

Init	result = 1, loop starts at i = 2	result = 1
i=2	result = 1 × 2 = 2	result = 2
i=3	result = 2 × 3 = 6	result = 6
i=4	result = 6 × 4 = 24	result = 24
i=5	result = 24 × 5 = 120 → loop ends	result = 120 ✓

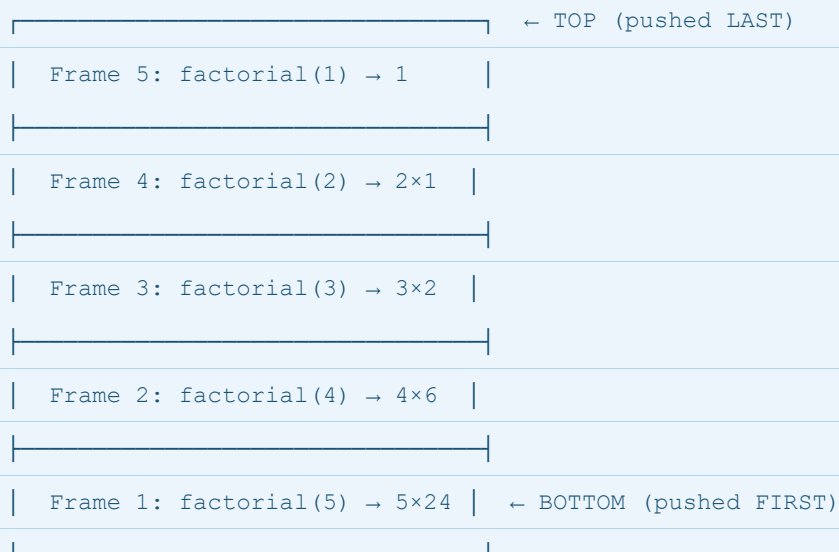
PQ3	Using the concept of a stack frame, explain the performance of the recursive method using Big O notation.	[10]
------------	--	-------------

Answer — Stack Frames Explained

Every time a method calls itself, Java allocates a NEW STACK FRAME on the call stack. Each frame stores:

- The return address (where to go after returning)
- The method parameters (the value of n)
- Local variables for that call

Call Stack for factorial(5):



Each frame = one method call = one unit of stack memory.

For factorial(n) → exactly n frames are open simultaneously.

→ Space Complexity = O(n)

Each frame performs O(1) work (one multiplication + one return).

Total work across all n frames = $n \times O(1) = O(n)$

→ Time Complexity = $O(n)$

PQ4 Compare the performance of recursive vs iterative factorial. Which is preferable in the real world?

[5]

	RECURSIVE	ITERATIVE
Time Complexity	$O(n)$ — n recursive calls	$O(n)$ — n loop iterations
Space Complexity	$O(n)$ — n stack frames in memory	$O(1)$ — one result variable only
Stack Risk	Can cause <code>StackOverflowError</code> for large n	None — no stack usage
Code Readability	Clean, mirrors maths formula	More verbose but straightforward
Real World?	❌ Avoid for simple problems like factorial	✅ Preferred — memory-efficient

Conclusion — Marks Answer

ITERATIVE is preferable in the real world for factorial because:

1. Space: $O(1)$ vs $O(n)$ — no risk of stack overflow no matter how large n is.
2. Speed: No method-call overhead — just a single loop variable.
3. Scalability: Works for $n = 1,000,000$ without crashing.

Use recursion ONLY when the problem is naturally recursive (e.g. tree traversal, divide & conquer).

PQ5 Determine the Big O expression for the three code excerpts.

[9]

Excerpt A

```
int sum = 0;

for (int i = 0; i < n; i++) {    // Loop 1: runs n times

    sum = sum + a[i];

}

for (int j = 0; j < n; j++) {    // Loop 2: runs n times
```

```
sum = sum + a[j];
```

```
}
```

Answer: $O(n)$

Loop 1 runs n times = $O(n)$

Loop 2 runs n times = $O(n)$

Total = $O(n) + O(n) = O(2n)$

Rule: Drop the constant $\rightarrow O(n) \leftarrow$ FINAL ANSWER

Sequential loops ADD their complexities, then constants are dropped.

Excerpt B

```
int prod = 1;
```

```
for (int i = 1; i < n; i++) {           // Outer: runs n times
```

```
    for (int j = i; j < n; j = j*3) { // Inner: j triples  $\rightarrow \log_3(n)$  steps
```

```
        prod = prod * j;
```

```
    }
```

```
}
```

Answer: $O(n \log n)$

Outer loop: i goes $1, 2, 3, \dots, n \rightarrow$ runs n times.

Inner loop: j goes $i, i \times 3, i \times 9, \dots \rightarrow$ multiplies by 3 each time = $\log_3(n)$ steps.

Total = $O(n) \times O(\log n) = O(n \log n)$

Rule: Nested loops MULTIPLY. A tripling loop is logarithmic.

Excerpt C

```
int sum = 0;
```

```
for (i = 0; i <= n-1; i += 3) { // Outer: steps of 3  $\rightarrow n/3$  iterations
```

```
    for (j = n-1; j > 0; j--) { // Inner: counts down from  $n$  to 1  $\rightarrow n$  steps
```

```
        sum = sum + (i * j);
```

```
    }
```

```
}
```

Answer: $O(n^2)$

Outer loop: $i = 0, 3, 6, \dots, n-1 \rightarrow$ runs $n/3$ times.

Inner loop: $j = n-1, n-2, \dots, 1 \rightarrow$ runs n times.

Total = $O(n/3) \times O(n) = O(n^2/3)$

Rule: Drop the $1/3$ constant $\rightarrow O(n^2) \leftarrow$ FINAL ANSWER

Even though the outer loop only runs $n/3$ times, constants vanish in Big O.

PQ6 Show how Bubble Sort modifies {90, 23, 5, 109, 12} step-by-step.

[12]

How Bubble Sort Works

Compare neighbouring elements. If left > right, swap them. Repeat until no swaps in a full pass. The largest element "bubbles up" to the end after each pass.

Trace

Step	Action / Explanation	Array / State
P1.1	Compare 90 & 23 $\rightarrow 90 > 23 \rightarrow$ SWAP	[23, 90, 5, 109, 12]
P1.2	Compare 90 & 5 $\rightarrow 90 > 5 \rightarrow$ SWAP	[23, 5, 90, 109, 12]
P1.3	Compare 90 & 109 $\rightarrow 90 < 109 \rightarrow$ No swap	[23, 5, 90, 109, 12]
P1.4	Compare 109 & 12 $\rightarrow 109 > 12 \rightarrow$ SWAP $\rightarrow$ 109 in place $\checkmark$	[23, 5, 90, 12, 109]
P2.1	Compare 23 & 5 $\rightarrow 23 > 5 \rightarrow$ SWAP	[5, 23, 90, 12, 109]
P2.2	Compare 23 & 90 $\rightarrow 23 < 90 \rightarrow$ No swap	[5, 23, 90, 12, 109]
P2.3	Compare 90 & 12 $\rightarrow 90 > 12 \rightarrow$ SWAP $\rightarrow$ 90 in place $\checkmark$	[5, 23, 12, 90, 109]
P3.1	Compare 5 & 23 $\rightarrow 5 < 23 \rightarrow$ No swap	[5, 23, 12, 90, 109]
P3.2	Compare 23 & 12 $\rightarrow 23 > 12 \rightarrow$ SWAP $\rightarrow$ 23 in place $\checkmark$	[5, 12, 23, 90, 109]
P4.1	Compare 5 & 12 $\rightarrow 5 < 12 \rightarrow$ No swap $\rightarrow$ DONE $\checkmark$	[5, 12, 23, 90, 109]

Case	Complexity	Reason
Best Case	$O(n)$	Already sorted — 1 pass, no swaps (with swapped flag)

Average Case	$O(n^2)$	Random order — roughly $n^2/2$ comparisons
Worst Case	$O(n^2)$	Reverse sorted — max comparisons: $n(n-1)/2$

PQ7	Show how Insertion Sort modifies {90, 23, 5, 109, 12} step-by-step.	[12]
------------	--	-------------

How Insertion Sort Works

Pick the next unsorted element. Slide it LEFT through the sorted portion until it is in the right position. Like sorting playing cards in your hand.

```
public static void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];          // Element to insert
        int j = i - 1;

        // Shift elements that are greater than key one position right
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;          // Insert key at correct position
    }
}
```

Trace

Step	Action / Explanation	Array / State
i=1	key=23. Compare 90>23 → shift 90 right. Insert 23 before 90.	[23, 90, 5, 109, 12]
i=2	key=5. Compare 90>5 → shift. Compare 23>5 → shift. Insert 5 at front.	[5, 23, 90, 109, 12]
i=3	key=109. Compare 90<109 → No shift. 109 stays in place.	[5, 23, 90, 109, 12]
i=4	key=12. Compare 109>12 → shift. Compare 90>12 → shift. Compare 23>12 → shift. Compare 5<12 → stop. Insert 12.	[5, 12, 23, 90, 109]
Done	All elements sorted ✓	[5, 12, 23, 90, 109]

Case	Complexity	Reason
Best Case	$O(n)$	Already sorted — inner while-loop never executes
Average Case	$O(n^2)$	Random data — shifts needed on average $n/2$ times per element
Worst Case	$O(n^2)$	Reverse sorted — maximum shifts every iteration

PQ8	Show how Quick Sort (middle pivot) modifies {9,7,5,11,12,2,14,3,10,6}.	[12]
-----	--	------

How Quick Sort Works

1. Choose PIVOT (middle element here).
2. PARTITION: elements $<$ pivot go LEFT, elements $>$ pivot go RIGHT.
3. Recursively sort left and right sub-arrays.

```

public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);    // Sort left half
        quickSort(arr, pivotIndex + 1, high);    // Sort right half
    }
}

public static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];    // (Using last element as pivot in this code)

    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;

            // swap arr[i] and arr[j]
            int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
        }
    }

    // Place pivot in correct position
    int temp = arr[i+1]; arr[i+1] = arr[high]; arr[high] = temp;

```

```
return i + 1;
```

```
}
```

Trace — Array: {9, 7, 5, 11, 12, 2, 14, 3, 10, 6}

Middle element (index 4) = 12 → Pivot = 12

Step	Action / Explanation	Array / State
R1	Full array. Middle index = $(0+9)/2 = 4$, Pivot = 12.	[9, 7, 5, 11, 12, 2, 14, 3, 10, 6]
R1a	Partition: Elements < 12: {9,7,5,11,2,3,10,6} 12 Elements > 12: {14}	[9, 7, 5, 11, 2, 3, 10, 6 12 14]
R2	Left sub {9,7,5,11,2,3,10,6}: middle index=3, Pivot=11	[9, 7, 5, 11, 2, 3, 10, 6]
R2a	Partition: < 11: {9,7,5,2,3,10,6} 11 > 11: {}	[9, 7, 5, 2, 3, 10, 6 11]
R3	Left sub {9,7,5,2,3,10,6}: middle=3, Pivot=2	[9, 7, 5, 2, 3, 10, 6]
R3a	Partition: < 2: {} 2 > 2: {9,7,5,3,10,6}	[2 9, 7, 5, 3, 10, 6]
R4	Sub {9,7,5,3,10,6}: Pivot=3. < 3: {} 3 > 3: {9,7,5,10,6}	[2, 3 9, 7, 5, 10, 6]
R5	Sub {9,7,5,10,6}: Pivot=5. < 5: {} 5 > 5: {9,7,10,6}	[2, 3, 5 9, 7, 10, 6]
R6	Sub {9,7,10,6}: Pivot=7. < 7: {6} 7 > 7: {9,10}	[2, 3, 5, 6, 7 9, 10]
R7	Sub {9,10}: Pivot=9. < 9: {} 9 > 9: {10}	[2, 3, 5, 6, 7, 9, 10]
Done	Combine all → SORTED ✓	[2, 3, 5, 6, 7, 9, 10, 11, 12, 14]

Case	Complexity	Reason
Best Case	$O(n \log n)$	Pivot always splits array into equal halves
Average Case	$O(n \log n)$	Random data gives roughly balanced partitions
Worst Case	$O(n^2)$	Pivot is always smallest or largest element

TEST 1 — 10 MAY 2021 (80 Marks) — Full Answers & Code

Q1.1 Create a program to implement a BST where students are sorted by name.

[20]

Concept: Binary Search Tree (BST)

A BST is a tree where: left child < parent < right child. For strings (names), this means alphabetical order. Searching, inserting and deleting are all $O(\log n)$ on average.

```
// — Student class —————  
  
class Student {  
    String name;  
  
    String studentId;  
  
    double overallMark;  
  
    public Student(String name, String studentId, double overallMark) {  
        this.name      = name;  
  
        this.studentId = studentId;  
  
        this.overallMark = overallMark;  
  
    }  
}  
  
// — BST Node —————  
  
class BSTNode {  
    Student data;  
  
    BSTNode left, right;  
  
    public BSTNode(Student data) {  
        this.data = data;  
  
        this.left = null;  
  
        this.right = null;  
  
    }  
}  
  
// — BST class —————
```

```

class StudentBST {

    private BSTNode root;

    public StudentBST() {

        root = null;

    }

    // Insert a student into the BST (sorted by name alphabetically)
    public void insert(Student student) {

        root = insertRecursive(root, student);

    }

    private BSTNode insertRecursive(BSTNode node, Student student) {

        if (node == null) {

            return new BSTNode(student);    // Empty spot found – insert here

        }

        int cmp = student.name.compareTo(node.data.name);

        if (cmp < 0) {

            node.left = insertRecursive(node.left, student); // Go left

        } else if (cmp > 0) {

            node.right = insertRecursive(node.right, student); // Go right

        }

        // cmp == 0 means duplicate name – ignore

        return node;

    }

    // Inorder traversal → prints students in alphabetical order
    public void inorder() {

        inorderRecursive(root);

    }

    private void inorderRecursive(BSTNode node) {

        if (node == null) return;           // Base case

```

```

        inorderRecursive(node.left);          // Visit LEFT subtree first

        System.out.println(                  // Visit CURRENT node

            "Name: " + node.data.name +

            ", ID: " + node.data.studentId +

            ", Mark: " + node.data.overallMark

        );

        inorderRecursive(node.right);        // Visit RIGHT subtree
    }

}

// — Main —————
public class Main {

    public static void main(String[] args) {

        StudentBST tree = new StudentBST();

        tree.insert(new Student("John",    "21034569",  67));

        tree.insert(new Student("Ayanda",  "23245432",  98));

        tree.insert(new Student("Khaya",   "214354233", 76));

        tree.insert(new Student("Zanele",  "21087654",  82));

        System.out.println("Students in alphabetical order:");

        tree.inorder();

        // Output:

        // Name: Ayanda, ID: 23245432, Mark: 98.0

        // Name: John,   ID: 21034569, Mark: 67.0

        // Name: Khaya,  ID: 214354233, Mark: 76.0

        // Name: Zanele, ID: 21087654, Mark: 82.0

    }

}

```

Q2.1	Explain the working principle and time performance of BST insert and inorder traversal.	[10]
-------------	--	-------------

INSERT METHOD	INORDER TRAVERSAL
----------------------	--------------------------

- Start at the root node.
- Compare the new value with the current node.
- If new < current → go LEFT subtree.
- If new > current → go RIGHT subtree.
- If current node is NULL → insert here.

Time: $O(\log n)$ average, $O(n)$ worst case (skewed tree).

Visits nodes in the order: LEFT → ROOT → RIGHT.

- Recursively visit entire LEFT subtree.
- Process the CURRENT (root) node.
- Recursively visit entire RIGHT subtree.

Result: nodes are printed in ASCENDING (alphabetical) order.

Time: $O(n)$ — every node visited exactly once.

Q2.2 Explain the working principle and time performance of the circular queue algorithm.

[10]

What is a Queue?

A queue is a FIFO (First In, First Out) data structure. The first item added is the first item removed — like a supermarket queue.

What makes it Circular?

A regular queue wastes space when items are removed from the front. A circular queue treats the array as a ring — when the rear pointer reaches the end, it wraps around to the beginning.

```
class CircularQueue {
    private int[] queue;

    private int front, rear, size, capacity;

    public CircularQueue(int capacity) {
        this.capacity = capacity;
        this.queue = new int[capacity];
        this.front = 0;
        this.rear = 0;
        this.size = 0;
    }

    // ENQUEUE — add to rear

    public void enqueue(int value) {
        if (size == capacity) {
            System.out.println("Queue is FULL");
            return;
        }

        queue[rear] = value;
```

rear = (rear + 1) % capacity; // Wrap around using modulo
size++;
}
// DEQUEUE - remove from front
public int dequeue() {
if (size == 0) {
System.out.println("Queue is EMPTY");
return -1;
}
int value = queue[front];
front = (front + 1) % capacity; // Wrap around using modulo
size--;
return value;
}
}
// Time Complexity: O(1) for both enqueue and dequeue
// Space Complexity: O(n) where n = capacity

Key Insight: The Modulo Trick	
rear = (rear + 1) % capacity	
When rear reaches the last index (capacity-1), adding 1 and taking modulo wraps it back to 0.	
Example (capacity=5): rear=4 $\rightarrow (4+1)\%5 = 0 \rightarrow$ wraps to front of array.	
This is what makes it "circular". Time complexity of each operation = O(1).	

Q3.1	Implement a stack using a linked representation. Stack Student objects.	[20]
------	---	------

Concept: Linked Stack

A stack is a LIFO (Last In, First Out) structure. A linked stack uses nodes (like a linked list) instead of an array, so it grows dynamically with no fixed size limit.

```
// — Student class —————  
  
class Student {  
    String name;  
  
    String studentId;  
  
    double overallMark;  
  
  
    public Student(String name, String studentId, double overallMark) {  
        this.name      = name;  
  
        this.studentId  = studentId;  
  
        this.overallMark = overallMark;  
  
    }  
  
    public String toString() {  
  
        return "Name: " + name + ", ID: " + studentId + ", Mark: " +  
overallMark;  
  
    }  
  
}
```

```
// — Stack Node —————  
  
class StackNode {  
  
    Student data;  
  
    StackNode next;    // Points to the node below in the stack  
  
  
    public StackNode(Student data) {  
  
        this.data = data;  
  
        this.next = null;  
  
    }  
  
}
```

```
// — Linked Stack —————  
  
class LinkedStack {  
  
    private StackNode top;    // Points to the top of the stack  
  

```

```

public LinkedStack() {

    top = null;

}

// PUSH - add to top  (O(1))
public void push(Student student) {

    StackNode newNode = new StackNode(student);

    newNode.next = top; // New node points to old top

    top = newNode;      // Top is now the new node

    System.out.println("Pushed: " + student);

}

// POP - remove from top  (O(1))
public Student pop() {

    if (isEmpty()) {

        System.out.println("Stack is EMPTY");

        return null;

    }

    Student data = top.data;

    top = top.next;      // Move top down by one

    return data;

}

// PEEK - see top without removing  (O(1))
public Student peek() {

    if (isEmpty()) return null;

    return top.data;

}

public boolean isEmpty() {

    return top == null;

}

}

```

```
// — Main —————

public class Main {

    public static void main(String[] args) {

        LinkedList stack = new LinkedList();

        // Push three students

        stack.push(new Student("John", "21034569", 67));
        stack.push(new Student("Ayanda", "23245432", 98));
        stack.push(new Student("Khaya", "214354233", 76));

        // Stack state (top → bottom): Khaya → Ayanda → John

        System.out.println("Top: " + stack.peek());

        // Pop all

        System.out.println("Popped: " + stack.pop()); // Khaya (LIFO)
        System.out.println("Popped: " + stack.pop()); // Ayanda
        System.out.println("Popped: " + stack.pop()); // John

    }

}

/* Expected Output:

    Pushed: Name: John, ID: 21034569, Mark: 67.0
    Pushed: Name: Ayanda, ID: 23245432, Mark: 98.0
    Pushed: Name: Khaya, ID: 214354233, Mark: 76.0
    Top: Name: Khaya, ID: 214354233, Mark: 76.0
    Popped: Name: Khaya, ID: 214354233, Mark: 76.0
    Popped: Name: Ayanda, ID: 23245432, Mark: 98.0
    Popped: Name: John, ID: 21034569, Mark: 67.0

*/
```

Stack Memory Diagram (after 3 pushes)

TOP → [Khaya | 214354233 | 76.0] → [Ayanda | 23245432 | 98.0] → [John | 21034569 | 67.0] → NULL

Last pushed (Khaya) is first to be popped. That is LIFO.

Q4.1 Recursive factorial, Iterative factorial, Big O comparison.

[20]

/

Q4.2

/

Q4.3

Full code and trace is identical to Practice Questions PQ1, PQ2, PQ3, PQ4 above — please refer to those sections. Key answers:

Summary Answers

Q4.1 Recursive: factorialRecursive(n) — base case: $n \leq 1$ return 1, else return $n * \text{factorial}(n-1)$

Q4.2 Iterative: factorialIterative(n) — loop from $i=2$ to n , multiply result each time

Q4.3 Time complexity: BOTH are $O(n)$

Q4.3 Space complexity: Recursive = $O(n)$ stack frames | Iterative = $O(1)$

Q4.3 Real world: ITERATIVE is preferred — no stack overflow risk, $O(1)$ space.

TEST 1 — 7 MARCH 2024 — Full Answers & Code

Q1.1 Using a trace, explain how the **RECURSIVE** factorial method works.

[5]

Full trace shown in PQ1/PQ3 above. Here is a condensed answer suitable for exam:

Trace of factorial(5):
factorial(5) = 5 × factorial(4) ← calls itself with n-1
factorial(4) = 4 × factorial(3)
factorial(3) = 3 × factorial(2)
factorial(2) = 2 × factorial(1)
factorial(1) = 1 ← BASE CASE (n==1, return 1)
factorial(2) = 2 × 1 = 2
factorial(3) = 3 × 2 = 6
factorial(4) = 4 × 6 = 24
factorial(5) = 5 × 24 = 120 ← FINAL RESULT

Q1.2 Using a trace, explain how the **ITERATIVE** factorial method works.

[5]

Step	Action / Explanation	Array / State
Init	int result = 1; loop starts	result = 1
i=2	result = 1 × 2	result = 2
i=3	result = 2 × 3	result = 6
i=4	result = 6 × 4	result = 24
i=5	result = 24 × 5 → loop ends	result = 120 ✓

Q1.3 Using Big O, compare time and space performance of the two algorithms.

[6]

See PQ4 comparison table above. Summary:

	RECURSIVE	ITERATIVE
Time Complexity	O(n)	O(n)
Space Complexity	O(n) — n stack frames	O(1) — one variable

Q1.4	Which is preferable in the real world?	[4]
Answer		
ITERATIVE is preferable for factorial in the real world.		
Reason 1: Space — iterative uses $O(1)$ space vs $O(n)$ stack frames for recursive.		
Reason 2: Safety — recursive method risks <code>StackOverflowError</code> for large n .		
Reason 3: Efficiency — no method-call overhead for iterative.		
Note: Recursion is better for problems naturally modelled recursively (BST, divide & conquer).		

Q2.1	Show how QuickSort and HeapSort modify {7,5,15,20,16,17,8,3}. (Diagrams)	[14]
------	--	------

QuickSort Trace — {7, 5, 15, 20, 16, 17, 8, 3} (Last element as pivot)

Step	Action / Explanation	Array / State
R1	Pivot=3 (last). All others > 3. Partition: {} 3 {7,5,15,20,16,17,8}	[3 7, 5, 15, 20, 16, 17, 8]
R2	Sub {7,5,15,20,16,17,8}: Pivot=8. <8:{7,5} 8 >8:{15,20,16,17}	[3 5, 7 8 15, 16, 17, 20]
R3	Sub {7,5}: Pivot=5. <5:{} 5 >5:{7}	[3, 5, 7, 8 ...]
R4	Sub {15,20,16,17}: Pivot=17. <17:{15,16} 17 >17:{20}	[3, 5, 7, 8 15, 16, 17, 20]
R5	Sub {15,16}: Pivot=16. <16:{15} 16 >16:{}	[3, 5, 7, 8, 15, 16, 17, 20]
Done	All sub-arrays are size 1 → SORTED ✓	[3, 5, 7, 8, 15, 16, 17, 20]

HeapSort Trace — {7, 5, 15, 20, 16, 17, 8, 3}

PHASE 1 — Build Max-Heap (heapify from last non-leaf upward):

Initial array:	[7, 5, 15, 20, 16, 17, 8, 3]
As a binary tree:	
	7
	/ \
	5 15
	/ \ / \

20 16 17 8
/
3
Heapify from index 3 downward:
Index 3 (val=20): children 3(idx7). 20>3 → no swap needed.
Index 2 (val=15): children 17(idx5), 8(idx6). 17>15 → SWAP 15 and 17.
After: [7, 5, 17, 20, 16, 15, 8, 3]
Index 1 (val=5): children 20(idx3), 16(idx4). 20>5 → SWAP 5 and 20.
After: [7, 20, 17, 5, 16, 15, 8, 3]
Now check swapped node (idx3, val=5): children 3(idx7). 5>3 → no swap.
Index 0 (val=7): children 20(idx1), 17(idx2). 20>7 → SWAP 7 and 20.
After: [20, 7, 17, 5, 16, 15, 8, 3]
Check node(idx1, val=7): children 5(idx3), 16(idx4). 16>7 → SWAP 7 and 16.
After: [20, 16, 17, 5, 7, 15, 8, 3]
MAX-HEAP BUILT: [20, 16, 17, 5, 7, 15, 8, 3] (root=20 is max) ✓

PHASE 2 — Extract max and re-heapify:

Step	Action / Explanation	Array / State
E1	Swap root 20 with last (3). Remove 20.	[3,16,17,5,7,15,8 20]
E2	Re-heapify: 3 at root. 17>16→swap with 17. 15>3→swap with 15.	[17,16,15,5,7,3,8 20]
E3	Swap root 17 with last unsorted (8). Remove 17.	[8,16,15,5,7,3 17,20]
E4	Re-heapify. Extract 16 → ...	[3,7,15,5 16,17,20]
E5	Extract 15 → ...	[3,7,5 15,16,17,20]
E6	Extract 7 → ...	[3,5 7,15,16,17,20]
E7	Extract 5, then 3 → SORTED ✓	[3, 5, 7, 8, 15, 16, 17, 20]

Q2.2 – Q2.7	Complexity for QuickSort and HeapSort.	[6 (1 mark each)]
--------------------	---	--------------------------

	Case	QuickSort	HeapSort
Q2.2/Q2.3	Best	$O(n \log n)$	$O(n \log n)$
Q2.4/Q2.5	Average	$O(n \log n)$	$O(n \log n)$
Q2.6/Q2.7	Worst	$O(n^2)$	$O(n \log n)$

Q2.8	What do you mean by algorithm?	1
-------------	---------------------------------------	----------

Answer

An algorithm is a finite, ordered set of well-defined, unambiguous instructions that solves a specific problem.

It must have: (1) Input — zero or more inputs, (2) Output — at least one output,

(3) Definiteness — each step is clearly defined, (4) Finiteness — it terminates in a finite number of steps,

(5) Effectiveness — each step is feasible and executable.

Q2.9	What are the characteristics of an algorithm?	1
-------------	--	----------

Answer — 5 Characteristics

1. INPUT: Zero or more well-defined inputs.

2. OUTPUT: At least one result/output must be produced.

3. DEFINITENESS: Every step must be clear and unambiguous.

4. FINITENESS: Must terminate after a finite number of steps.

5. EFFECTIVENESS: Every operation must be basic enough to be carried out exactly.

Q2.10	What is the difference between time complexity and space complexity in Big O?	1
--------------	--	----------

TIME COMPLEXITY

Measures how the number of OPERATIONS grows as input size n grows.

SPACE COMPLEXITY

Measures how much MEMORY is used as input size n grows.

Examples:

$O(1)$ — constant time (array lookup)
 $O(n)$ — linear (single loop)
 $O(n^2)$ — quadratic (nested loops)
 $O(\log n)$ — logarithmic (binary search)

Examples:

$O(1)$ — constant space (iterative factorial)
 $O(n)$ — linear (recursive stack frames, storing n items)
 $O(n^2)$ — storing a 2D matrix of size $n \times n$

Q2.11 How does a recursive method work in Java?

A recursive method in Java is a method that calls itself. Java uses the JVM call stack to manage this.

```
// When factorial(3) is called, Java creates stack frames:
```

```
Stack Frame 3 (top):    factorial(1) → returns 1
```

```
Stack Frame 2:          factorial(2) → returns 2 * 1 = 2
```

```
Stack Frame 1 (bottom): factorial(3) → returns 3 * 2 = 6
```

```
// Each frame holds: return address, parameter value, local variables.
```

```
// When factorial(1) hits the base case, it returns 1,
```

```
// then frame 3 is popped, frame 2 computes 2*1=2, and so on.
```

```
// If there is no base case, Java throws: java.lang.StackOverflowError
```

TEST 1 — 3 APRIL 2025 (75 Marks) — Full Answers & Code

Q1 Determine the Big O expression for each code excerpt.

[9]

Full worked solutions are in Practice Questions PQ5 above. Summary:

#	Code Description	Answer & Reason
A	Two sequential loops, each runs n times	$O(n)$ — $O(n)+O(n) = O(2n) \rightarrow$ drop constant $\rightarrow O(n)$
B	Outer loop n times; inner loop: j tripled each step	$O(n \log n)$ — outer $O(n) \times$ inner $O(\log n)$
C	Outer loop steps of 3 ($n/3$ times); inner counts down n times	$O(n^2)$ — $O(n/3) \times O(n) = O(n^2/3) \rightarrow$ drop constant $\rightarrow O(n^2)$

Q2.1 Demonstrate how recursive factorial code executes. Report on time and space complexity.

□

Full trace: see PQ1 and Q1.1 (2024) above. Code is identical. Summary:

```
public static int factorial(int n) {
    if (n == 0 || n == 1) return 1;        // BASE CASE
    return n * factorial(n - 1);          // RECURSIVE CASE
}

// Execution for n=4:
// factorial(4) → 4 × factorial(3)
//   factorial(3) → 3 × factorial(2)
//     factorial(2) → 2 × factorial(1)
//       factorial(1) → returns 1  (BASE CASE)
//     factorial(2) = 2 × 1 = 2
//   factorial(3) = 3 × 2 = 6
// factorial(4) = 4 × 6 = 24  ✓
```

Q2.2 Report on the space and time complexity of factorial in Big O.

□

Answer

Time Complexity (Recursive): $O(n)$ — exactly n method calls, each $O(1)$ work.

Space Complexity (Recursive): $O(n)$ — n stack frames open simultaneously in JVM memory.

Time Complexity (Iterative): $O(n)$ — loop runs $n-1$ times.

Space Complexity (Iterative): $O(1)$ — only one variable "result" used regardless of n .

Q3.1 Show how Bubble Sort modifies {90, 23, 5, 109, 12}.



Full trace: see Practice Question PQ6 above. All 4 passes traced step-by-step.

Quick Summary

Pass 1 end: [23, 5, 90, 12, 109] — 109 bubbled to end

Pass 2 end: [5, 23, 12, 90, 109] — 90 in place

Pass 3 end: [5, 12, 23, 90, 109] — 23 in place

Pass 4: No swaps → SORTED: [5, 12, 23, 90, 109] ✓

Q3.2 Show and explain the Big O expression for Bubble Sort.



```
// Bubble Sort (with optimised swapped flag):  
  
public static void bubbleSort(int[] arr) {  
    int n = arr.length;  
  
    for (int i = 0; i < n - 1; i++) {           // Outer: n-1 passes  
        boolean swapped = false;  
  
        for (int j = 0; j < n - 1 - i; j++) {    // Inner: n-1-i comparisons  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
                swapped = true;  
            }  
        }  
  
        if (!swapped) break; // Already sorted - exit early → Best case  $O(n)$   
    }  
}
```

```

}

// Big O Analysis:

// Outer loop: n-1 iterations

// Inner loop: (n-1) + (n-2) + ... + 1 = n(n-1)/2 total comparisons

// Drop constants and lower order terms:

//  $n(n-1)/2 \approx n^2/2 \rightarrow O(n^2)$ 

```

Case	Complexity	Reason
Best Case	$O(n)$	Array already sorted — swapped flag triggers early exit after 1 pass
Average Case	$O(n^2)$	Random data — roughly $n^2/2$ comparisons on average
Worst Case	$O(n^2)$	Reverse sorted — maximum $n(n-1)/2$ comparisons

Q3.3	Show how Quick Sort modifies {90, 23, 5, 109, 12}.	📄
------	--	---

Full trace: see Section 5.1 in Master Guide / PQ8 above. Summary:

Step	Action / Explanation	Array / State
R1	Pivot=12 (last). < 12: {5} 12 > 12: {90,23,109}	[5 12 90,23,109]
R2	Left {5}: size 1 → sorted ✓	[5]
R3	Sub {90,23,109}: Pivot=109. < 109:{90,23} 109 > 109:{}	[90,23 109]
R4	Sub {90,23}: Pivot=23. <23:{} 23 >23:{90}	[23, 90]
Done	Combine all → SORTED ✓	[5, 12, 23, 90, 109]

Q3.4	Show and explain the Big O expression for Quick Sort.	📄
------	---	---

// Big O for Quick Sort:
Best Case: $O(n \log n)$
→ Pivot splits array into two equal halves every time.

→ $\log n$ levels of recursion $\times$ n work per level = $O(n \log n)$

Average Case: $O(n \log n)$

→ Pivot gives roughly balanced partitions on average for random data.

Worst Case: $O(n^2)$

→ Pivot is always the smallest or largest element.

→ Happens when array is already sorted and pivot = last element.

→ One partition always has 0 elements, other has $n-1$.

→ n levels deep, n work each = $O(n^2)$

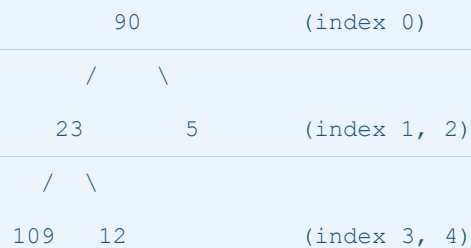
// To avoid worst case: use random pivot or median-of-three pivot strategy.

Q3.5 Show how Heap Sort modifies {90, 23, 5, 109, 12}.



Full trace: see Section 6.2 in Master Guide above. Summary:

Array as tree:



Phase 1 – Build Max-Heap:

Heapify index 1 (val=23): children 109 and 12. $109 > 23 \rightarrow$ SWAP 23 and 109.

Array: [90, 109, 5, 23, 12]

Heapify index 0 (val=90): children 109 and 5. $109 > 90 \rightarrow$ SWAP 90 and 109.

Array: [109, 90, 5, 23, 12] → MAX-HEAP ✓ (109 = max at root)

Phase 2 – Extract max:

Step 1: Swap 109 (root) with 12 (last). Remove 109.

Array: [12, 90, 5, 23 | 109] → Re-heapify → [90, 23, 5, 12 | 109]

Step 2: Swap 90 (root) with 12. Remove 90.
Array: [12, 23, 5 90, 109] → Re-heapify → [23, 12, 5 90, 109]
Step 3: Swap 23 with 5. Remove 23.
Array: [5, 12 23, 90, 109] → Re-heapify → [12, 5 23, 90, 109]
Step 4: Swap 12 with 5. Remove 12.
Array: [5 12, 23, 90, 109]
Done: [5, 12, 23, 90, 109] ✓

Q3.6	Show and explain the Big O expression for Heap Sort.	
// Big O for Heap Sort:		
Phase 1 – Build Max-Heap: $O(n)$		
→ Although heapify for one node is $O(\log n)$, across all n nodes = $O(n)$ total.		
Phase 2 – n extractions: $O(n \log n)$		
→ Each extraction (swap + re-heapify) costs $O(\log n)$		
→ n extractions $\times O(\log n) = O(n \log n)$		
Total: $O(n) + O(n \log n) = O(n \log n)$		
Best Case: $O(n \log n)$ ← SAME for all cases		
Average Case: $O(n \log n)$		
Worst Case: $O(n \log n)$ ← Never degrades to $O(n^2)$ unlike QuickSort		
Space Complexity: $O(1)$ – sorts in-place, no extra array needed.		
Case	Complexity	Reason
Best Case	$O(n \log n)$	Same as all cases — tree operations are always $\log n$
Average Case	$O(n \log n)$	Consistent regardless of input
Worst Case	$O(n \log n)$	Guaranteed — no bad pivot problem like QuickSort

FINAL QUICK-REFERENCE — Complexity Summary

Algorithm	Best	Average	Worst	Space	Stable
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	—
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	—
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗
BST Insert	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$	—
BST Inorder	$O(n)$	$O(n)$	$O(n)$	$O(n)$	—
Circular Queue	$O(1)$	$O(1)$	$O(1)$	$O(n)$	—
Stack Push/Pop	$O(1)$	$O(1)$	$O(1)$	$O(n)$	—

Good luck on Thursday Thabiso — all the best! 🍀 ADP470S 2026